Troubleshooting Dynamic Collection Issues

Contents

I. Introduction
II. General Information
III. Common Troubleshooting Steps
IV. Common Scenarios
V. Examples
VI. Reference information

I. Introduction

The cornerstone of EM7 is data collection. If an individual collection is failing to collect, there are 3 basic questions to consider:

- What type of collection is responsible for the data?
- How does EM7 retrieve the information?
- Is EM7 failing to collect, or is the device not sending the appropriate data?

II. General Information

What type of collection is responsible for the data?

EM7 has two basic types of collections: Internal collections and Dynamic Application collections. This guide focuses on Dynamic Application collection, but when troubleshooting, you need to first determine whether dynamic applications are used to collect the data.1

Internal Collections:

Internal Collections generally include data that has common collection methods across most physically monitored devices, as well as custom Monitors (as defined in Registry>Monitors). Internal Collections include things like:

- Interface bandwidth collection and alerting
- Availability
- Latency

- System uptime
- Process and service collection
- File System discovery and utilization
- Domain Name, Email Round-Trip, SOAP-XML, SSL Certificate, System Process, TCP-IP Port, Web Content, and Windows Service Monitors

Dynamic Collections

Dynamic Applications are how the majority of data is collected for devices monitored by EM7, and provides the method for extending core collection functionality. Dynamic applications are used to collect data using methods that are more specific to certain types of devices or manufacturers. Common Dynamic Application collections include:

- Hardware Configuration
- Application performance
- CPU, Memory, and Swap utilization
  Note: Though EM7 groups certain collections as "System Vitals" (including CPU, Memory, and Swap utilization), the collection methods and OIDs vary widely for these collections and the actually collections are performed by dynamic applications, such as "Net-SNMP: Physical Memory"

If you are unsure if a particular data set is collected via an Internal Process or a Dynamic Application, find a device that contains the type of data you are looking for. If the data is collected via a Dynamic Application, you will see a dynamic app aligned to the device on the "Collections" tab of the Device Details page.

How Does EM7 Retrieve the Data?

Once you have determined if EM7 collects the data via Internal Process or dynamic application, you need to understand how EM7 goes about collecting the data. For specific types of Internal Collections, see the Guides or Manuals related to those collections.

To Determine How EM7 retrieves data, Navigate to System>Manage>Applications, and locate the application which collects the data you're looking for. This application type will identify the collection method:

- SNMP
- Database
- Powershell
- WMI
- SOAP
- XML
- XSLT
- Snippet

Common among all dynamic application is that they use collection objects which define how the application gets data using the collection type, eg: the SNMP OID retrieved, or the SQL query performed.

Wrench the application you are reviewing and click the collections tab. Select the collection object(s) for the data you are looking for. You can then review the particular collection used for the data. If your application is a SOAP, Snippet, or XSLT application, you will need to also review the "Requests" or "Snippets" tab for more details about how the data is collected.

Test Collection outside of the collection process

After determining how EM7 tries to collect the data, you should be able to test the collection method. This varies greatly depending on the type of application, but testing outside of the dynamic app collection process will help isolate the retrieved data from EM7 process to determine if it is a connection issue, device issue, or EM7 software/dynamic app issue.

It's always best to perform collection from the collector which will perform the collector. Often a port may be open on a device, but not between a collector and a device, due to a collector being beyond a firewall or network segment from the device which is to be monitored.

Some tests might include performing SQL queries or SNMP queries from the collector against the device. For example, for a device at ip address 10.0.0.1, you might SSH to the collector and run:

snmpwalk –v 2c –c public 10.0.0.1 <your collection oid>

or:

mysql –h 10.0.0.1 –u dbuser –p –e "Query from collection object;"

III. Common Troubleshooting Steps

Running collection in debug mode

1. The simplest method for debugging dynamic app collection is to navigate to the device in Registry>Devices>Device Manager. Click the wrench for the device, and click the Collections tab. Locate the dynamic app in question, and click the lightning bolt.
2. Debugging can be run from the command line of the collector responsible for the device with the command:
   dynamic_single.py
   To run this command, you need the device ID, DID and application ID, AID. From the collector, run the following:
   su - s-em7-core -c /usr/local/silo/proc/dynamic_single.py DID AID
3. If collection cannot be debugged using the above methods, or you need to observed the scheduled synchronous collection behavior, enable debugging for "Data Collection: Dynamic

App” under System>Settings>Processes. After collection has occurred, retrieve silo.log files from the collector

Performing Collection Independent of EM7

1. SNMP:
   snmpwalk -v 2c -c <community string> <hostname or IP> <OID>
   snmpgetnext -v 2c -c <community string> <hostname or IP> <OID>
2. SOAP/cURL
   (GET):
   curl -k -v -u <user> https://<hostname>/<URI>
   (POST JSON):
   curl -k -v -u <user> https://<hostname>/<URI> -H 'content-type:application/json' --data-binary @data.json
3. MySQL:
   mysql -h <hostname> -u<user> -p<password> -e "select <oid> from <database>.<table>;"

IV. Common Scenarios

Dynamic App not Collecting Data

Symptoms:

- Performance Graphs for a dynamic application are blank
- Config tables not displaying data snapshots
- Events not firing

Possible Causes:

- Device is in maintenance/user disabled/unavailable
- Invalid credential aligned to the dynamic app
- EM7 credential contains invalid details
- Device not responding to dynamic app
- Dynamic App is defective

Troubleshooting steps:

1. Run collection in debug mode from the “Collections” tab of the device
2. If the error “Dynamic App is not aligned to Device” occurs, confirm the device is in an available state
3. Confirm collection objects for the dynamic app are set to “collect – Yes”
4. Confirm the appropriate type of credential is aligned to the dynamic application
5. Run collection in debug from the command line, check for errors
6. Test collection outside of the collection process (SNMP get, SQL Query, cURL request, etc)

    a. Confirm the username/password provided by the customer are valid
    b. Confirm that valid data is returned when performing a request outside of EM7
    c. Re-enter the customer provided username and passwords into the credential under System>Manage>Credentials
7. Enable debugging for Dynamic App collection and retrieve debug logs once collection has run
8. Review logs for errors or other information

Dynamic App Collecting Incorrect Data

Symptoms:

- EM7 collected data does not match data from another tool or customer's observations
- EM7 not generating an event when a threshold is crossed, eg: high CPU not generating an event

Possible Causes:

- App aligned to the wrong device
- Device reporting incorrect data
- Device model not supported/Dynamic app defective

Troubleshooting steps:

- System>Manage>Credentials
    - check the host specified for the app credential. This is usually %D, which substitutes the device's hostname or IP address, but can also be set to a specific hostname or IP address. Make sure this is %D, or a valid IP address for the device
- Perform a traceroute from the collector to the device IP address.
    - Make sure the IP address is not being routed to an unexpected device. Sometimes collectors are in different subnets than devices and and traffic could be erroneously routed to the wrong device (this will require customer knowledge of the environment being monitored)
- From the collector, try to perform collection outside of the collection process.
    - Check the data returned to see if it matches the customer's expectation.

Dynamic App Not Generating Events

Symptoms:

- No events for expected issue with device

Possible Causes:

- App Generating no data or the wrong data (see above)

- Event Formula invalid
- Events disabled/suppressed

Troubleshooting steps

- Run collection in debug mode
    - This will show the data returned as well as the alert threshold and alert formula
- Check the device logs
    - Is there a log entried that is colored like an event, but no event ID? Check suppressions
- View the event policy in Registry>Events>Event Manager
    - Check to see if there are any event suppression on the Suppressions tab
    - Check the advanced tab
        - Is there an occurrence count requiring more than one alert to trigger the event?
        - Is there an occurrence time requiring a minimum amount of time for the event to be active

V. Examples

Example 1:

A customer who uses video monitoring power-packs calls and says they are not receiving CDR (Call Detail Records) journals for their Tandberg device. We wrench the device from the device registry, and navigate to the collections tab. From there, we see the CDR application for this device is "Cisco: Call Detail Records (Tandberg C Series)"

Dynamic Application™ Collections

Expand | Action | Reset | Guide

Dynamic Application	ID	Poll Frequency	Type	Credential
Cisco: Call Quality Statistics (Tandberg C Series)	428	1 mins	XSLT Performance	N/A
Cisco: Active Call (Tandberg C Series)	422	1 mins	XSLT Config	Tandberg Endpoint - QA - Config
Cisco: Audio/Video (Tandberg C Series)	423	5 mins	XSLT Config	Tandberg Endpoint - QA - Config
Cisco: Configuration (Tandberg C Series)	421	15 mins	XSLT Config	Tandberg Endpoint - QA - Config
Cisco: Configuration Cache (Tandberg C Series)	416	0 mins	XSLT Config	Tandberg Endpoint - QA - Config
Cisco: Remote Management (Tandberg C Series)	415	15 mins	XSLT Config	Tandberg Endpoint - QA - Config
Cisco: Status Cache (Tandberg C Series)	420	0 mins	XSLT Config	Tandberg Endpoint - QA- Status
Cisco: Call Detail Records (Tandberg C Series)	429	30 mins	Snippet Journal	Tandberg Endpoint - QA - History

We can already see that this is a snippet journal application. We look at the Journals tab and see that there appear to be no objects collected for the Call Detail Records app.

We go back to the collections tab and run the application in debug mode. Clicking the lightning bolt runs the application in debug. In this case, at the end of the debugging output, we see an error: Authentication Failed.

The customer insists the password is correct.

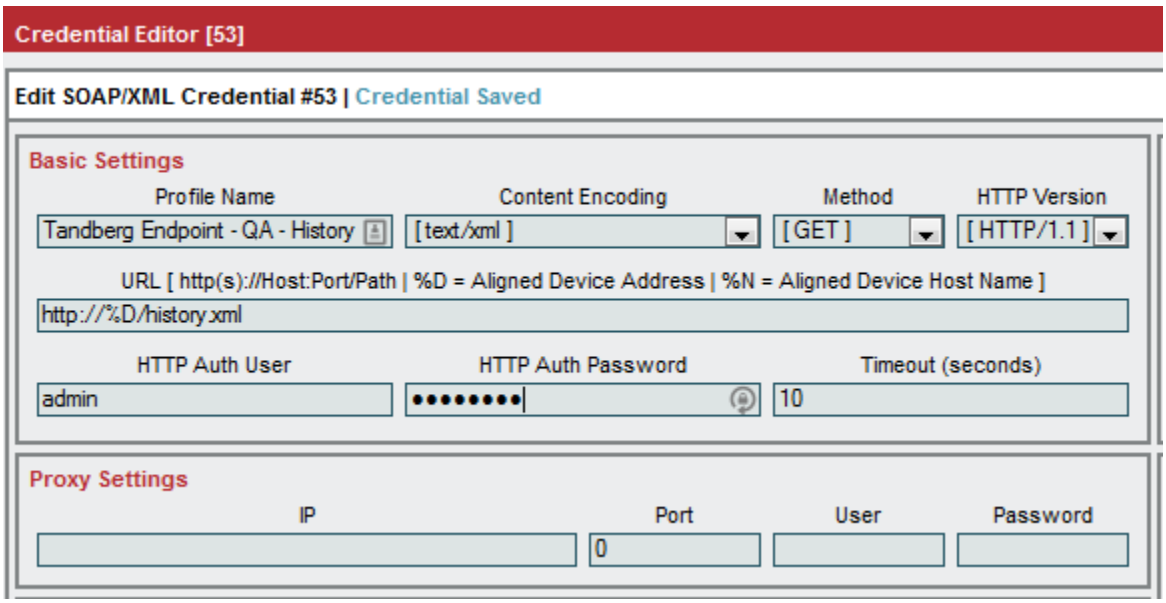
Navigating to System>Manage>Applications, we can take a look at the snippet to see how it is collecting. At the top of the snippet we see that the snippet is looking for 2 XML files, history.xml and status.xml.

```
app_name = "Tandberg C-Series CDR"
CRED_TYPE_XML = 3
DEFAULT_TIMEOUT = 5

HISTORY_XML = 'history.xml'
STATUS_XML = 'status.xml'
TIME_FMT = "%Y/%m/%d %H:%M:%S"

def get_call_key(call_info):
    if 'StartTime' in call_info:
        return call_info['StartTime']
```

Without even knowing how to read Python code, it's a safe bet that these 2 files will be downloaded from the device being monitored, and information about the device being monitored is passed to the dynamic application via the credential. So let's look at the credential. From the collections tab, we saw the credential aligned to this application.



This shows us where to find the history.xml file on the device, as well as the username being used to grab the XML. We should be able to load http://10.0.0.1/history.xml from our device, using the username 'admin' and the password the customer provides. The credential also shows that the request is a standard GET request. (Note: This may or may not be honored by a Snippet, but a properly built app should have the credential configured to match the collection method). In this case, we're getting an authentication error, not a connection error, so we can just try to load the XML file from our own web browser, instead of going to the collector and pulling it with cURL.

So we go to:

http://10.0.0.1/history.xml

At the login prompt, we ask the customer to enter the username ‘admin’ and the password. Authentication passes and the XML file is downloaded. The customer says “SEE! The password works!”.

We navigate back to the credential, and delete the password, and have the customer type the same password back into the credential editor, then save the credential.

Finally, we go back to the collections tab for the device and run collection in debug mode. Authentication is successful and journals are collected successfully. It turns out the customer had the right password, but the credential had the wrong password.

VI. Reference Information

Dynamic app data tables

Dynamic applications create a MySQL database for each dynamic application. The name of the database is always:

dynamic_app_data_<aid>

where <aid> is the Application ID number.

Data stored during collection is stored in one of several types of tables, depending on the application type. There will be one data table per device. In the three table definitions below, <DID> is the device ID of the device for which the table contains data.

Performance apps:

dev_stats_<DID>

Configuration apps:

dev_config_<DID>

Journal apps:

dev_journal_<DID>

Additionally, application data Dbs may contain the following tables, which contain meta information about the applications and the data normalized by the normalization processes:

app_label
app_meta
journal_meta
normalized_daily
normalized_freq
normalized_hourly
normalized_state

For example, data for Host Resource Memory with application ID 8 would store data for Device ID 105 in dynamic_app_data_8.dev_stats_105.

The normalized data would appear in:

dynamic_app_data_8.normalized_freq
dynamic_app_data_8.normalized_hourly

dynamic_app_data_8.normalized_daily
dynamic_app_data_8.normalized_state

Table Structure:

Performance app data tables have the following structure:

```
+-----------------+----------------------+------+-----+---------+-------+
| Field           | Type                 | Null | Key | Default | Extra |
+-----------------+----------------------+------+-----+---------+-------+
| object          | mediumint(3) unsigned | NO   | PRI | 0       |       |
| collection_time | datetime             | NO   | PRI | NULL    |       |
| data            | char(25)             | YES  |     | NULL    |       |
| ind             | int(11) unsigned     | NO   | PRI | 0       |       |
| elapsed         | smallint(5) unsigned | NO   |     | 0       |       |
```

Similarly, configuration app data tables have the following structure:

```
+-----------------+----------------------+------+-----+---------+-------+
| Field           | Type                 | Null | Key | Default | Extra |
+-----------------+----------------------+------+-----+---------+-------+
| object          | mediumint(3) unsigned | NO   | PRI | 0       |       |
| collection_time | datetime             | NO   | PRI | NULL    |       |
| data            | char(128)            | YES  |     | NULL    |       |
| ind             | int(11) unsigned     | NO   | PRI | 0       |       |
| grp             | tinyint(3) unsigned  | NO   | PRI | 0       |       |
| changed         | tinyint(3) unsigned  | NO   | MUL | 0       |       |
| elapsed         | smallint(5) unsigned | NO   |     | 0       |       |
```